

FUNCTIONS IN PYTHON

FUNCTION

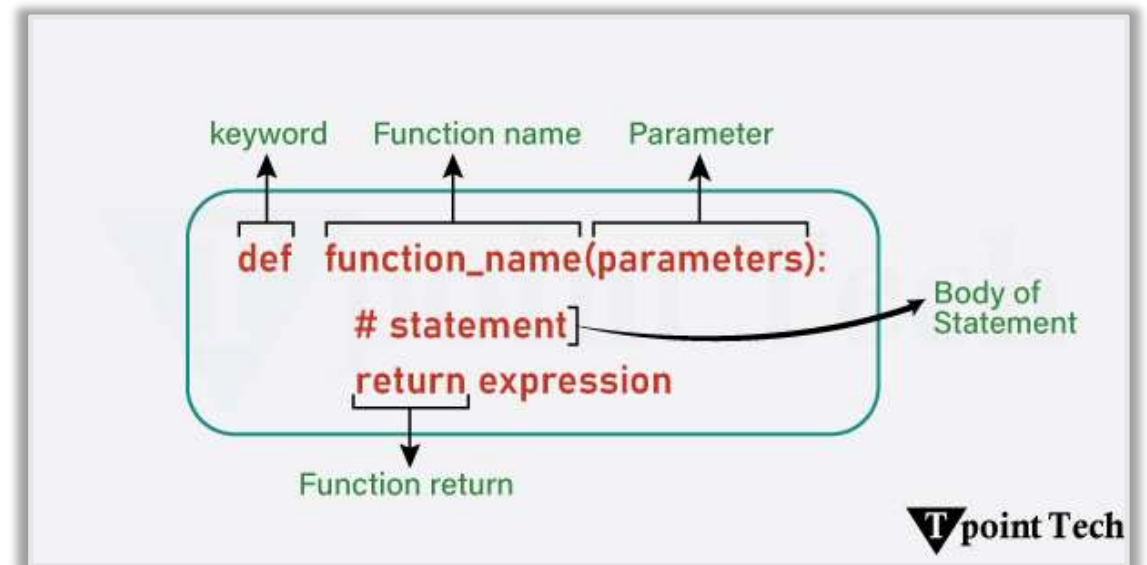
A named block of reusable code that performs a specific task.

- only runs when it is called
- Pass data
- return data as a result

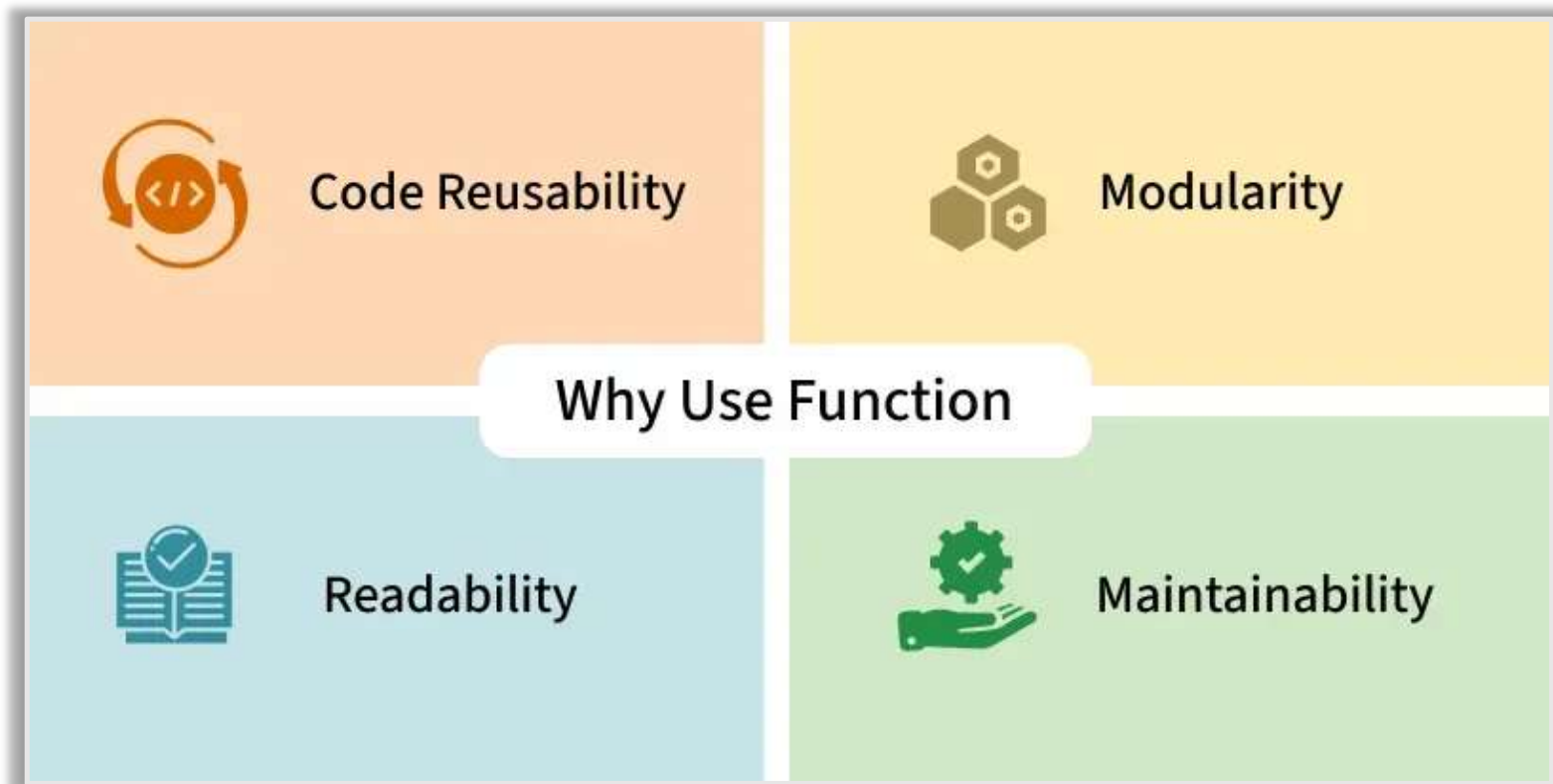
Syntax : **def function-name(parameters):**

#statement

return expression



Why use functions ?



CALLING A FUNCTION

Syntax: fun-name ()

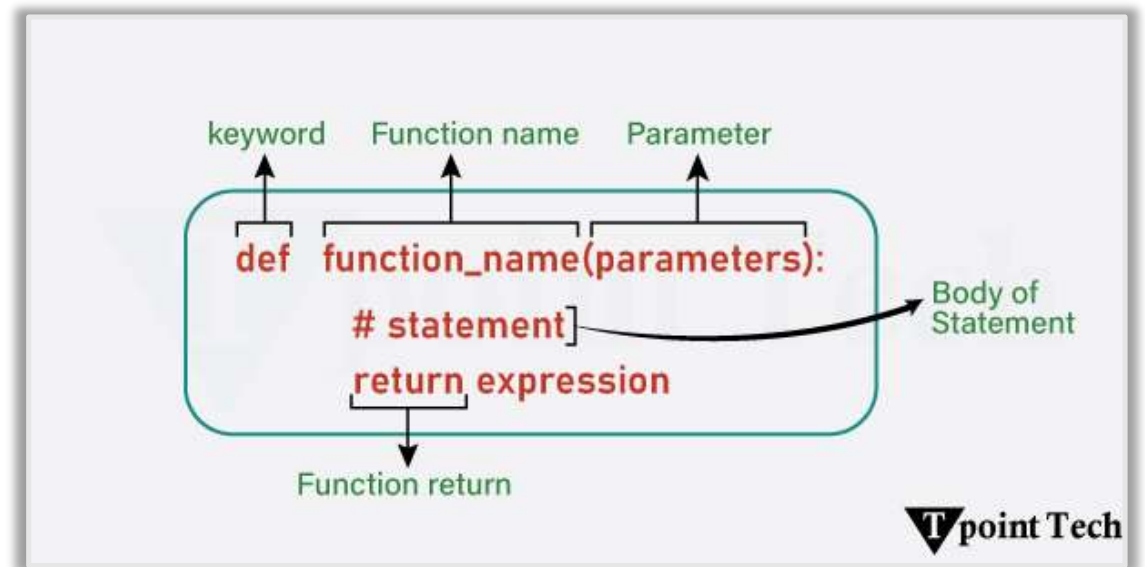
Example:

Defining a Function:

```
def greet( ):
    print("Welcome to python")
```

#Calling a function –

greet () #function name followed by parenthesis



FUNCTION WITH ARGUMENTS

Syntax:

def function_name(parameters):

#body of the function

return expression

Function_name(arguments)

Parameters

- ❑ **Variables defined in the function header.**
- ❑ They are **placeholders** that receive values when the function is called.
- ❑ Declared when defining a function.

```
def greet( name ): # name is a parameter  
    print(“Welcome ”,name)
```

```
greet(“Quest”) # “Quest is an argument”
```

FUNCTION WITH ARGUMENTS

Arguments

- Arguments are the **actual values** you pass to a function when calling it.
- They replace the parameters.

```
def add(a, b): # a and b are parameters  
    return a + b
```

```
add(5, 10) # 5 and 10 are arguments
```

Quick Memory Trick

- **Parameters = Placeholders**
- **Arguments = Actual Values**

TYPE OF FUNCTION ARGUMENTS

1. POSITIONAL ARGUMENTS

- ✓ **Order matters**
- ✓ Values are assigned **based on position**, not name.

Example:

```
def details(name, age):  
    print("Name:", name)  
    print("Age:", age)  
  
details("Alice", 30)  
details(25, "Bob")
```

TYPE OF FUNCTION ARGUMENTS

2. KEYWORD ARGUMENTS

- ✓ Arguments passed using parameter names.
- ✓ Order doesn't matter.

Example:

```
def details(name, age):  
    print("Name:", name)  
    print("Age:", age)
```

```
details(age=30, name="Alice")
```

TYPE OF FUNCTION ARGUMENTS

3. DEFAULT ARGUMENTS

- ✓ Parameter with a predefined values
- ✓ If no value is provided during the function call, the default value is used automatically.
- ✓ The argument becomes **optional** when calling the function

Example:

```
def details(name, age=25):  
    print("Name:", name)  
    print("Age:", age)
```

```
details("Alice")
```

```
details("Bob", 30)
```

TYPE OF FUNCTION ARGUMENTS

4. ARBITRARY ARGUMENTS(VARIABLE LENGTH POSITIONAL ARGUMENTS) - *args

Allow a function to accept **any number of positional arguments**, even if you don't know how many will be passed in advance.

Syntax:

```
def function_name(*args):  
    # function body
```

***** → tells Python to accept any number of arguments.

args → A tuple of all passed positional arguments.

Note ↱ : can use any name instead of args, but args is the convention.

TYPE OF FUNCTION ARGUMENTS

4. ARBITRARY KEYWORD ARGUMENTS(****kwargs**)

Allow a function to accept **any number of keyword arguments**, even if you don't know the parameter names in advance.

Syntax :

```
def function_name(**kwargs):  
    # function body
```

****** → tells Python to accept **any number of keyword arguments**

kwargs → stores all keyword arguments in a **dictionary**

DIFFERENCE B/W return AND print()

Return	Print()
Sends a value back from a function	Displays output on the screen
Used inside a function	Can be used anywhere
Ends the function immediately	Does NOT stop the function
Output can be stored in a variable	Output cannot be stored (returns None)
Useful for calculations and further processing	Useful only for showing information to the user
Does not show anything on screen	Shows the value directly

Ways To Pass Values To A Function

Pass by Value

- ❑ The function gets a **copy** of the data. If the function changes that copy, the original variable outside the function stays the same.
- ❑ **Analogy:** You give a friend a photocopy of a map. They can draw on their copy, but your original map remains clean.
- ❑ **Common in:** C, Java (primitives), and Python (for numbers/strings).

Pass by Reference

- ❑ The function gets a **reference** (a pointer) to the original data. If the function modifies it, the original data changes too.
- ❑ **Analogy:** You give a friend the key to your house. If they paint the walls, the house is changed for you, too.
- ❑ **Common in:** C++ (using `&`), and JavaScript/Python (when passing objects or lists).

Pass by Object Reference / Assignment

- ❑ **Python is NOT truly pass by value and NOT truly pass by reference**
- ❑ In Python, the way we "pass values" is unique and often called **Pass by Object Reference** (or sometimes "Pass by Assignment").
- ❑ Instead of choosing between passing a value or a reference, Python's behavior depends entirely on whether the object you are passing is **mutable** (can be changed) or **immutable** (cannot be changed).
- ❑ When you pass objects like **integers, strings, or tuples**, Python behaves like **Pass by Value**. Because these objects cannot be changed, any "modification" inside the function actually creates a brand-new object.
- ❑ When you pass objects like **lists, dictionaries, or sets**, Python behaves like **Pass by Reference**. The function gets access to the actual object in memory. If you modify the object (e.g., adding an item to a list), that change persists outside the function.

Pass by Object Reference / Assignment

- It is important to note that **reassigning** a variable is different from **modifying** it. If you pass a list but then reassign the variable inside the function (e.g., `my_list = [1, 2, 3]`), you have broken the link to the original list, and the outside variable will not change.
- **Pro-tip:** If you want to pass a list to a function but ensure the function *cannot* change your original data, pass a copy instead: `my_function(my_list.copy())`.

Type Hinting

- Type hinting is a way to **specify the expected data type**
- Helps developers understand **what type of value a variable, function, or parameter should have**
- Introduced in **Python 3.5 (PEP 484)**
- **Why use Type Hinting?**
 - **Autocomplete:** Your editor (like VS Code or PyCharm) will suggest string methods (like `.upper()`) because it knows `name` is a string.
 - **Error Prevention:** Tools like `mypy` can scan your code and warn you if you're accidentally passing a list where a number should be.
 - **Documentation:** It serves as "built-in" documentation so you don't have to guess what a function needs.

DIFFERENT TYPE OF FUNCTIONS IN PYTHON

1. **BUILD-IN**
2. **USER DEFINED**
3. **RECURSIVE**
4. **LAMBDA**
5. **HIGHER ORDER**



1. BUILT-IN FUNCTIONS

- ❑ Predefined functions in Python
- ❑ available for use without importing any module or writing any code.
- ❑ come bundled with the Python interpreter.
- ❑ can be used directly in any Python program.

Examples of Built-in Functions

- ❑ `print()` – prints output
- ❑ `len()` – returns the length of a sequence
- ❑ `type()` – returns the type of a value
- ❑ `input()` – reads user input
- ❑ `max()` / `min()` – find largest/smallest value
- ❑ `sum()` – returns the sum of values
- ❑ `range()` – generates a sequence of numbers

2. USER DEFINED FUNCTIONS

- ❑ created by the programmer using the def keyword to perform a specific task.
- ❑ Defined using the def keyword.
- ❑ Can take arguments and return values.
- ❑ Help break code into smaller, manageable parts

Example:

```
def greet(name):  
    print("Hello",  
        name)
```

3. RECURSIVE FUNCTION

- ❑ A function that **calls itself**.
- ❑ A recursive function **must** have a **base case** to stop recursion.
- ❑ Without a base case, recursion will continue infinitely and cause an error.
- ❑ Used for problems like factorial, Fibonacci, tree traversal, etc.

❑ Example: Factorial Using Recursion

```
def factorial(n):  
    if n == 1:      # base case  
        return 1  
    return n * factorial(n - 1) #  
recursive call
```

4. LAMBDA FUNCTION

- ❑ A lambda function = a small one-line function without a name.
- ❑ meaning it has **no name** and is created using the lambda keyword.
- ❑ When Lambda Function Is Used
 1. The function is **very short**
 2. You need it **only one time**
 3. You don't want to define a full function with a name
- ❑ A lambda function has:
 - ◆ **Any number of arguments**
 - ◆ **ONLY one expression**

Syntax: **lambda arguments: expression**

Example:

```
f = lambda x: x + 5  
print(f(10))
```

5. HIGHER ORDER FUNCTIONS

- Functions that work with other functions.
- a function that **either takes another function as an argument, or returns a function as a result, or both.**
- Accept functions as input.
- Return functions as output.
- Used in Python functions like map(), filter(), reduce(), and decorators

□ Example 1: Passing a Function as Argument

```
def square(x):
```

```
    return x * x
```

```
def apply_func(func, value):
```

```
    return func(value)
```

```
print(apply_func(square, 5)) # Output: 25
```

5. HIGHER ORDER FUNCTIONS

1. `map()`

- `map()` is a function that applies another function to every item in a list (or any iterable).
- `map()` = “Do this function to every item.”

Example:

```
numbers = [1, 2, 3, 4]
```

```
result = map(lambda x: x * 2, numbers)
```

```
print(list(result))
```

5. HIGHER ORDER FUNCTIONS

2. filter()

filter() = “Keep only the items that pass the test.”

- It takes:

- a function that returns **True** or **False**

- a list (or iterable)

- and returns:

- only the items for which the function returned **True**

Example:

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
result = filter(lambda x: x % 2 == 0,  
numbers)
```

```
print(list(result))
```

5. HIGHER ORDER FUNCTIONS

3. reduce()

reduce() = “Combine all items into one result.”

takes a list (or any iterable) and reduces it to a single value by applying a function again and again to the items.

It works like this:

- ❑ Take the first two items → apply the function
- ❑ Take the result → apply the function with the next item
- ❑ Continue until only **one value** is left

Example:

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(lambda x, y: x + y, numbers)
```

```
print(result)
```

5. HIGHER ORDER FUNCTIONS

4. Decorator

- ❑ Without altering the actual code of a function, a decorator changes its behavior.
- ❑ does NOT modify the original function's code.
- ❑ wraps the function inside another function (wrapper)
- ❑ adds extra behavior before or after the function runs.
- ❑ returns a modified version of the function

Syntax:

```
def decorator_name(function_name):  
    def wrapper():  
        # code before  
        function_name()  
        # code after  
    return wrapper
```

```
@decorator_name  
def function_name():  
    # original function code  
    pass
```

DOCSTRINGS IN PYTHON

- A **special string** written inside a function, class, or module to explain **what it does**.
- Docstrings are written using **triple quotes** (""" """)
- They are placed **immediately after the function definition line**.
- You can access them using the function's `__doc__` attribute
- Why Docstrings Are Important?
 - Improve code readability
 - Help others understand your function
 - Required for professional projects
 - Used by tools like **help()**, IDE tooltips, and documentation generators

```
def add(a, b):  
    """  
    This function adds two numbers and returns the result.  
  
    Parameters:  
        a (int/float): First number  
        b (int/float): Second number  
  
    Returns:  
        int/float: Sum of a and b  
    """  
    return a + b  
  
print(add.__doc__)
```

FUNCTION SCOPE IN PYTHON

- ❑ **Scope = Where a variable is visible and usable /**
- ❑ **scope determines where a variable is accessible (visible) and where it can be read or modified.**
- ❑ Python follows the **LEGB** scope rule:

Scope	Meaning	Example
L – Local	Inside the current function	Variables defined inside a function
E – Enclosed	Inside outer functions (in nested functions)	Variables from an outer function used in an inner function
G – Global	At the top level of a program	Variables defined outside any function
B – Built-in	Predefined names in Python	print, len, sum, etc.

NESTED FUNCTIONS (INNER FUNCTIONS)

A function written inside another function.

The inner function can access variables of the outer function due to **enclosed scope**.

Syntax:

```
def outer_function():  
    def inner_function():  
        # inner function body  
        pass  
    inner_function() # calling inner function
```

```
def greet():  
    message = "Hello"  
  
    def show_message():  
        print(message) # accessing outer variable  
  
    show_message()
```

CLOSURE

A closure is an inner function that remembers variables from the outer function, even after the outer function has finished executing.

Closures are not just a theory—they are used in many real-world scenarios.

- ❑ **Why Closures Are Important?**
- ❑ They allow **data hiding**
- ❑ Help create **function factories**
- ❑ Used in **decorators**
- ❑ Keep state without using global variables

```
def outer():  
    x = 10  # outer variable  
  
    def inner():  
        print(x)  # inner remembers x → closure  
  
    return inner
```

```
func = outer()  # outer() has finished  
func()          # still prints 10 → closure
```

✓ The inner() function **remembers** the value of x, even though outer() is no longer running.

MULTIPLE RETURN VALUES

- ❑ Function can return **more than one value at the same time**.
- ❑ Python does this by **packing the values into a tuple**, even if you don't write the tuple explicitly.

```
def calculate(a, b):  
    return a + b, a * b
```

```
sum_val, product_val = calculate(4, 5)  
print(sum_val)      # 9  
print(product_val)  # 20
```